Experimental Evaluation of Real-Time Multi-Block Gaussian Volume Rendering

Author Name

1 Experimental Objectives

This section evaluates a multi-block Gaussian volume renderer in which a perspective camera is located at the centre of a moving cubic region. The camera supports six degrees of freedom, while rendering is spatially restricted to the moving cube. Independently pretrained Gaussian blocks are loaded when their spatial extents intersect the cube, and only Gaussian support intersecting the cube is retained for rendering.

The experiments are organised as a sequence of dependent milestones. Each milestone establishes the validity of the next stage. The evaluation addresses the following questions:

Q1: How does steady-state rendering frame rate change with active Gaussian payload?

Q2: What performance penalty is introduced by one-, two-, four-, and eight-block overlap configurations?

Q3: How much latency is introduced when the moving cube enters a new spatial region?

Q4: Which hardware-normalised metrics best predict whether the renderer sustains at least 30 FPS?

The milestones follow the progression

correctness → rendering scalability → multi-block overlap → streaming latency → cross-hardware deployment

2 System Definition

2.1 Gaussian Representation

Each Gaussian primitive is parameterised by a mean $\boldsymbol{\mu}_i \in \mathbb{R}^3$, anisotropic scale $\mathbf{s}_i \in \mathbb{R}^3$, orientation quaternion $\mathbf{q}_i$, and intensity I_i. The continuous scalar field is

$$V(\mathbf{x}) = \sum_{i=1}^{N} I_i \exp\left[-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_i)^\top \mathbf{Q}_i (\mathbf{x}-\boldsymbol{\mu}_i)\right], \tag{1}$$

where $\mathbf{Q}_i = \boldsymbol{\Sigma}_i^{-1}$ is the Gaussian precision matrix.

For a camera ray

$$\mathbf{r}(t) = \mathbf{c} + t\mathbf{d}, \tag{2}$$

the rendered maximum-intensity projection is

$$M(\mathbf{r}) = \max_{t\in[t_{\min},t_{\max}]} \sum_{i\in\mathcal{A}} I_i G_i(\mathbf{r}(t)), \tag{3}$$

where $\mathcal{A}$ is the active Gaussian set and $[t_{\min}, t_{\max}]$ is the intersection of the ray with the moving cube.

2.2 Moving Render Cube

Let the camera position be $\mathbf{c}$ and the cube half-edge be h. The moving render cube is

$$\mathcal{B}_s(\mathbf{c}) = [\mathbf{c} - h, \mathbf{c} + h]. \tag{4}$$

The moving cube determines:

1. which pretrained blocks are spatially relevant;
2. which Gaussian supports are active;
3. the exact ray interval used for rendering.

A pretrained block is loaded if its axis-aligned bounding box intersects $\mathcal{B}_s$. A Gaussian is active if its truncated support intersects $\mathcal{B}_s$.

Using a conservative support sphere, the radius of Gaussian i is approximated as

$$r_i = \sqrt{\tau}\max(s_{ix}, s_{iy}, s_{iz}), \tag{5}$$

where τ is the Mahalanobis cutoff. The Gaussian is retained when this support sphere intersects the moving cube.

2.3 Camera Motion

The camera position is fixed at the centre of the moving cube and is represented by

$$\mathbf{p} = (x, y, z). \tag{6}$$

Its orientation is controlled by yaw, pitch, and roll:

$$\boldsymbol{\theta} = (\theta_{\text{yaw}}, \theta_{\text{pitch}}, \theta_{\text{roll}}). \tag{7}$$

Translation modifies the set of intersecting blocks. Rotation changes the view frustum and tile assignment, but does not change block residency unless the camera position also changes.

3 Evaluation Metrics

3.1 Rendering Performance

Steady-state frame rate is defined as

$$\text{FPS} = \frac{1000}{T_{\text{render}}}, \tag{8}$$

where T_{render} is measured in milliseconds using CUDA events.

The real-time criterion is

$$\text{FPS} \geq 30, \tag{9}$$

or equivalently

$$T_{\text{render}} \leq 33.3 \text{ ms}. \tag{10}$$

For each configuration, we report the mean, median, standard deviation, minimum, fifth percentile FPS, and 95th percentile frame time.

3.2 Payload Metrics

The raw active Gaussian payload is

$$B_{\text{active}} = N_{\text{active}} B_{\text{record}}, \tag{11}$$

where N_{active} is the active Gaussian count and B_{record} is the number of bytes per Gaussian record.

The hardware-normalised active payload ratio is

$$R_{\text{active}} = \frac{B_{\text{active}}}{M_{\text{GPU}}}, \tag{12}$$

where M_{GPU} is total physical GPU memory.

We additionally define

$$R_{\text{loaded}} = \frac{B_{\text{loaded blocks}}}{M_{\text{GPU}}}, \tag{13}$$

and

$$R_{\text{renderer}} = \frac{B_{\text{all renderer allocations}}}{M_{\text{GPU}}}. \tag{14}$$

The renderer allocation includes Gaussian storage, tile keys and values, scan buffers, radix-sort buffers, tile ranges, output buffers, and temporary CUDA allocations.

3.3 Candidate Pressure

The average candidate pressure is

$$P_{\text{cand}} = \frac{N_{\text{Gaussian-tile pairs}}}{N_{\text{tiles}}}. \tag{15}$$

The average number of touched tiles per active Gaussian is

$$O_{\text{tile}} = \frac{N_{\text{Gaussian-tile pairs}}}{N_{\text{active}}}. \tag{16}$$

These metrics distinguish two payloads containing the same number of Gaussians but different spatial support and tile overlap.

3.4 Transition Latency

For a cold transition, the total new-area latency is

$$T_{\text{transition}} = T_{\text{read}} + T_{\text{transform}} + T_{\text{cube cull}} + T_{\text{H2D}} + T_{\text{bin rebuild}}. \tag{17}$$

For the current process-based implementation, the measured approximation is

$$T^{\text{cold}}_{\text{transition}} = T_{\text{block read/transform}} + T_{\text{cube cull/merge}} + T_{\text{renderer setup}}. \tag{18}$$

Transition bandwidth is

$$BW_{\text{transition}} = \frac{B_{\text{new payload}}}{T_{\text{transition}}}. \tag{19}$$

4 Milestone 1: Correctness and Instrumentation

4.1 Objective

The first milestone verifies that the moving-cube renderer is numerically correct and that timing and memory measurements are reliable.

4.2 M1.1: Single-Block Equivalence

A single pretrained block is rendered using both:

1. the original single-block inside-out renderer;
2. the moving-cube renderer with the cube fully contained in the same block.

Camera pose, field of view, image resolution, depth samples, Mahalanobis cutoff, and intensity activation are held constant.

The rendered outputs are compared using

$$\mathrm{MSE} = \frac{1}{N_p} \sum_{p=1}^{N_p} \left(I_p - \hat{I}_p \right)^2, \tag{20}$$

$$\mathrm{PSNR} = 10 \log_{10} \left(\frac{I_{\max}^2}{\mathrm{MSE}} \right), \tag{21}$$

$$E_{\max} = \max_p |I_p - \hat{I}_p|. \tag{22}$$

The acceptance targets are

$$\mathrm{PSNR} \geq 40 \text{ dB} \tag{23}$$

and

$$E_{\max} < 0.02. \tag{24}$$

Table 1: Single-block equivalence results.

Configuration	MSE	PSNR	SSIM	Max Error	Output Min	Output Max
TODO: Fill	1.718183e-05	47.649	0.9997	0.094	0.045	1.000

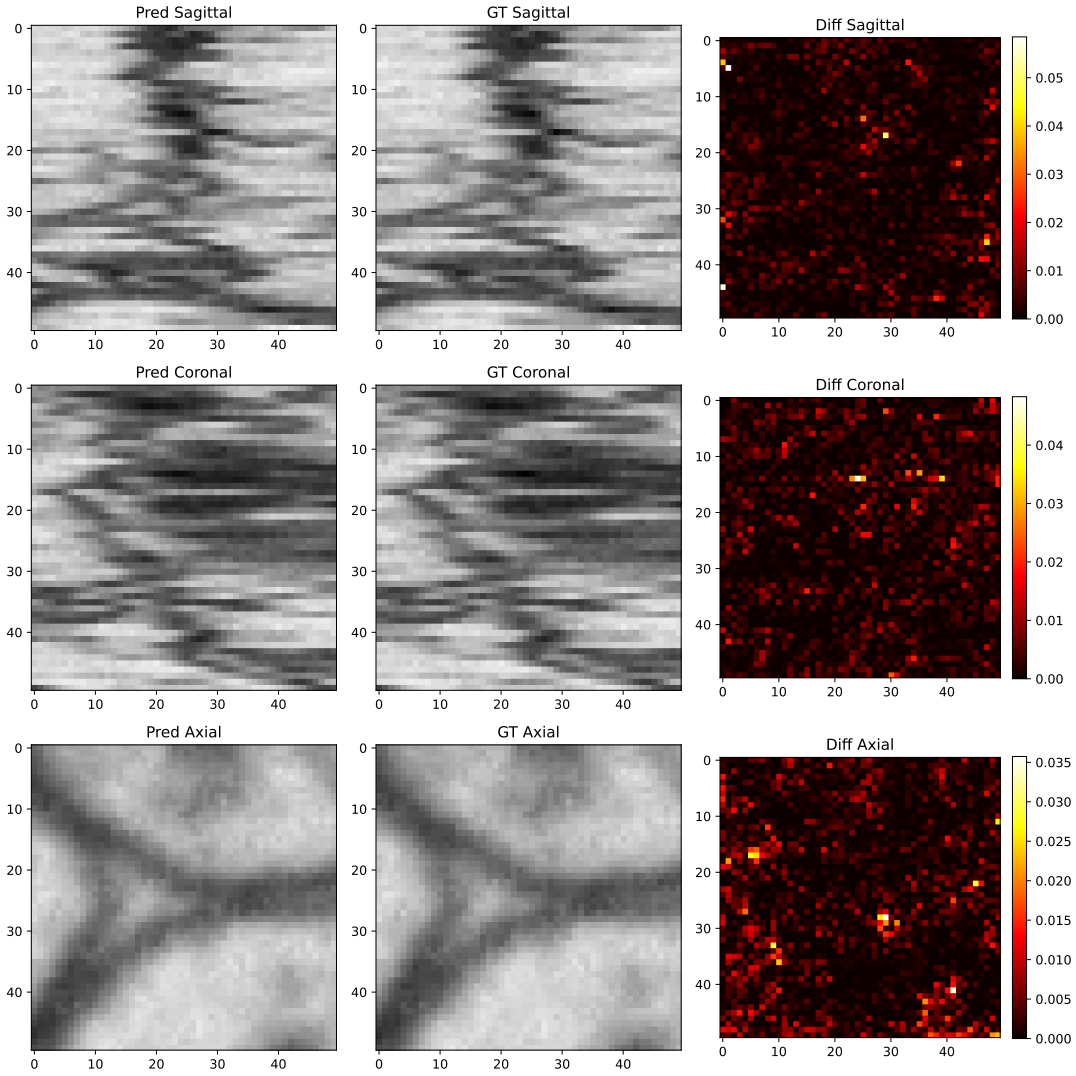

Figure 1: Single-block equivalence between the original renderer and the moving-cube renderer.

4.3 M1.2: Two-Block Boundary Correctness

Two adjacent pretrained blocks are placed along the x-axis. The camera-centred cube is translated through their shared boundary. At the midpoint, the cube contains approximately half of each block.

The dynamic renderer is compared against an offline merged reference containing both blocks permanently transformed into world coordinates.

Evaluation positions are

$$x \in \{x_0, x_0 + 0.25L, x_0 + 0.50L, x_0 + 0.75L, x_0 + L\}, \tag{25}$$

where L is the block edge length.

Table 2: Two-block boundary correctness.

Position	Overlap	Active Blocks	Active Gaussians	PSNR	SSIM	Max Error	Seam Score
TODO: Fill	–	–	–	–	–	–	–

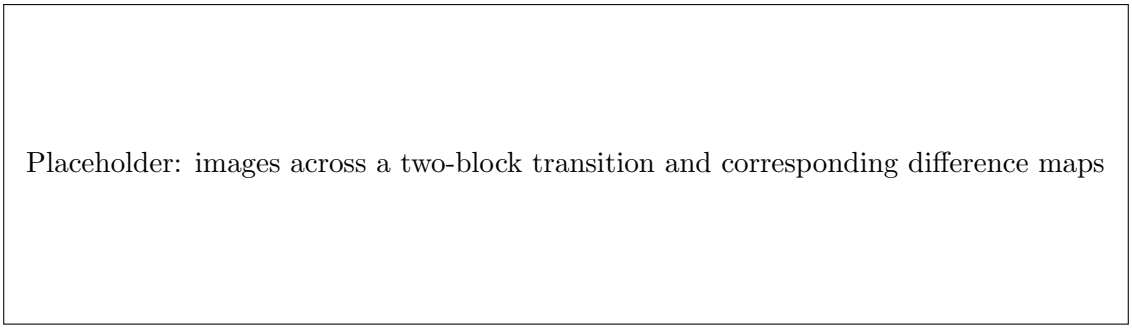

Figure 2: Qualitative continuity across the shared boundary of two independently pretrained blocks.

4.4 M1.3: Timing Validation

The CUDA event time is compared with CPU wall-clock time over a fixed benchmark interval. The difference should be attributable to process launch, input/output, synchronisation, and setup overhead.

Table 3: Timing instrumentation validation.

Configuration	CUDA Render Time	Wall Time	Setup Time	Difference	Relative Error
TODO: Fill	–	–	–	–	–

5 Milestone 2: Active Payload Versus FPS

5.1 Objective

The second milestone answers:

How does steady-state FPS change as the active Gaussian payload increases?

This experiment isolates payload scaling by keeping camera pose, moving-cube size, resolution, depth samples, field of view, and Mahalanobis cutoff fixed.

5.2 Controlled Payload Sweep

A dense camera location is selected. Starting from the complete active set, deterministic Gaussian subsets are constructed using retention ratios

$$\rho \in \{0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.75, 1.00\}. \tag{26}$$

The corresponding active set is

$$\mathcal{A}_\rho \subseteq \mathcal{A}_{1.0}. \tag{27}$$

A fixed random seed is used to ensure reproducibility.
The baseline configuration is:

Table 4: Baseline controlled payload configuration.

Parameter	Value
Resolution	**TODO:** e.g., 128×128
Depth samples	**TODO:** e.g., 64
Vertical field of view	**TODO:** e.g., 90°
Moving-cube edge length	**TODO:** Fill
Tile size	**TODO:** e.g., 16×16
Mahalanobis cutoff	**TODO:** e.g., 20
Warm-up frames	**TODO:** e.g., 20
Measured frames	**TODO:** e.g., 300
Independent repetitions	**TODO:** e.g., 5

5.3 Recorded Variables

For each payload level, the following quantities are recorded:

- active Gaussian count;
- active payload in MiB;
- R_{active};
- R_{renderer};
- Gaussian–tile pair count;
- P_{cand};
- O_{tile};
- median and p95 render time;
- median and p5 FPS;
- percentage of frames sustaining at least 30 FPS.

Table 5: Active payload scaling results.

Retention	Active Gaussians	Payload MiB	R_{active}	R_{renderer}	Pairs	P_{cand}	O_{tile}	Median FPS	p5 FPS	p95 Time
10%	–	–	–	–	–	–	–	–	–	–
20%	–	–	–	–	–	–	–	–	–	–
30%	–	–	–	–	–	–	–	–	–	–
40%	–	–	–	–	–	–	–	–	–	–
50%	–	–	–	–	–	–	–	–	–	–
60%	–	–	–	–	–	–	–	–	–	–
75%	–	–	–	–	–	–	–	–	–	–
100%	–	–	–	–	–	–	–	–	–	–

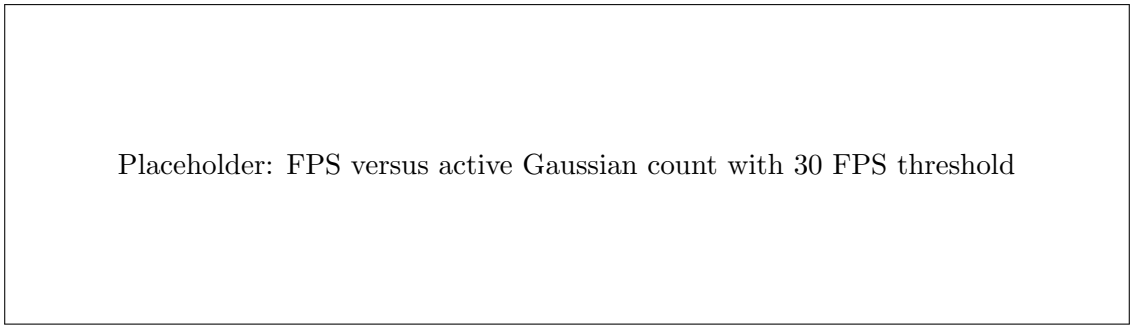

Figure 3: Rendering frame rate as a function of active Gaussian count.

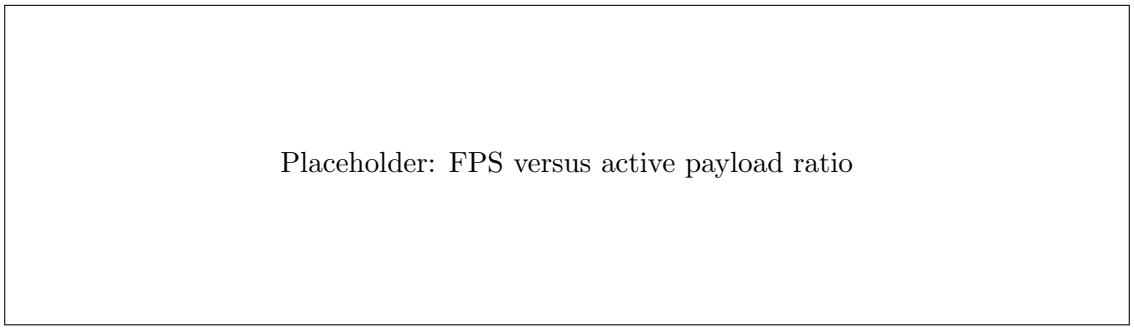

Figure 4: Rendering frame rate as a function of active Gaussian payload normalised by GPU memory.

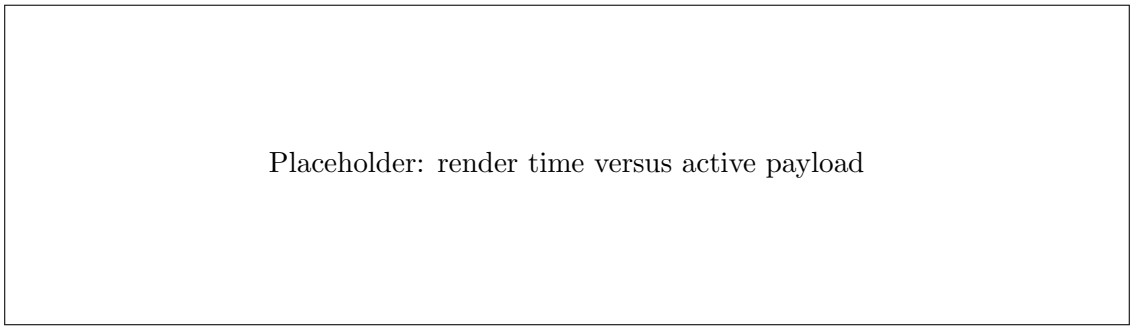

Figure 5: Render time as a function of active Gaussian payload.

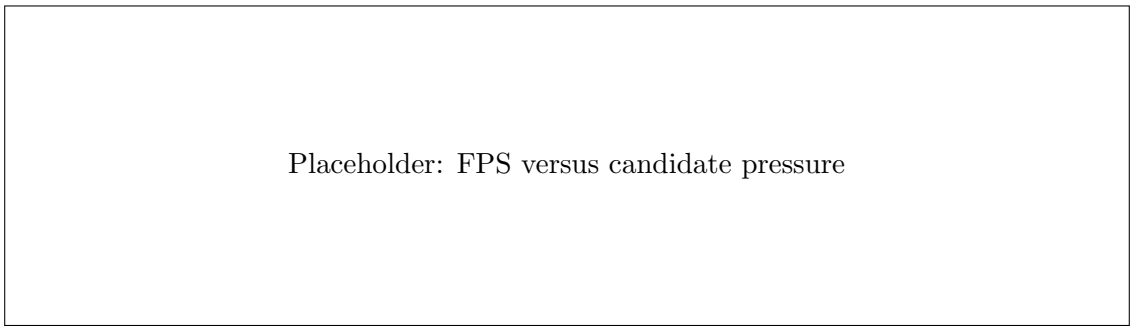

Figure 6: Rendering frame rate as a function of average Gaussian–tile candidate pressure.

5.4 Regression Analysis

The payload-only model is

$$T_{\text{render}} = \beta_0 + \beta_1 N_{\text{active}} + \epsilon. \tag{28}$$

The candidate-pressure model is

$$T_{\text{render}} = \beta_0 + \beta_1 P_{\text{cand}} + \epsilon. \tag{29}$$

The combined model is

$$T_{\text{render}} = \beta_0 + \beta_1 R_{\text{active}} + \beta_2 P_{\text{cand}} + \beta_3 O_{\text{tile}} + \epsilon. \tag{30}$$

Models are compared using adjusted R^2, RMSE, AIC, and residual analysis.

Table 6: Regression models for predicting render time.

Model	Predictors	R^2	Adjusted R^2	RMSE	AIC	Best
Payload only	N_{active}	–	–	–	–	–
Payload ratio	R_{active}	–	–	–	–	–
Candidate pressure	P_{cand}	–	–	–	–	–
Combined	$R_{\text{active}}, P_{\text{cand}}, O_{\text{tile}}$	–	–	–	–	–

5.5 Milestone 2 Outcome

This milestone identifies:

1. the largest active Gaussian count sustaining at least 30 FPS;
2. the largest active payload ratio sustaining at least 30 FPS;
3. the largest candidate pressure sustaining at least 30 FPS;
4. whether payload or candidate pressure better predicts render time.

TODO: Insert quantitative summary after experiments.

6 Milestone 3: Multi-Block Overlap Scaling

6.1 Objective

The third milestone determines whether natural one-, two-, four-, and eight-block overlap configurations follow the payload-performance trend established in Milestone 2.

The evaluated configurations are:

1. one-block interior;
2. two-block face boundary;
3. four-block edge junction;
4. eight-block corner junction.

For each configuration, the moving cube has the same edge length, resolution, field of view, depth samples, and camera orientation. Only the spatial overlap pattern changes.

Table 7: Multi-block overlap configurations.

Case	Active Blocks	Loaded Gaussians	Active Gaussians	Payload MiB	Pair Count	FPS
One block	1	–	–	–	–	–
Two blocks	2	–	–	–	–	–
Four blocks	4	–	–	–	–	–
Eight blocks	8	–	–	–	–	–

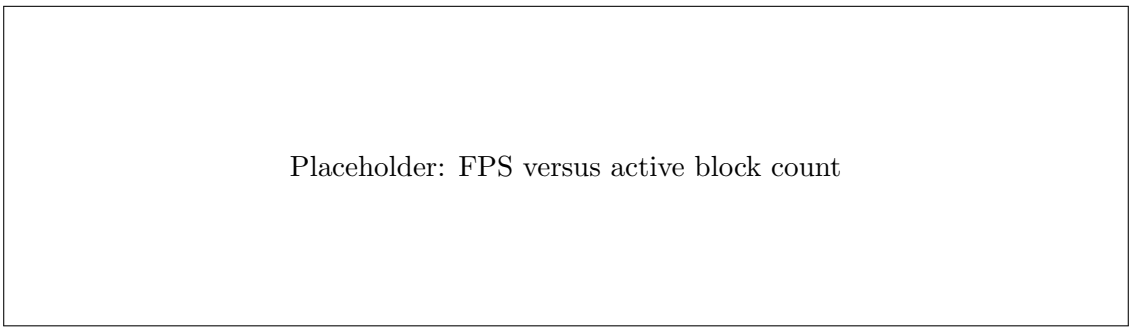

Figure 7: Frame rate for one-, two-, four-, and eight-block overlap configurations.

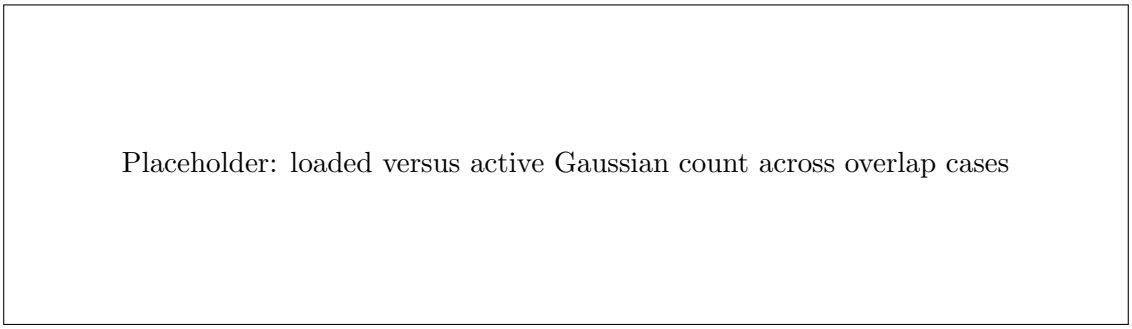

Figure 8: Loaded and active Gaussian populations for different spatial overlap patterns.

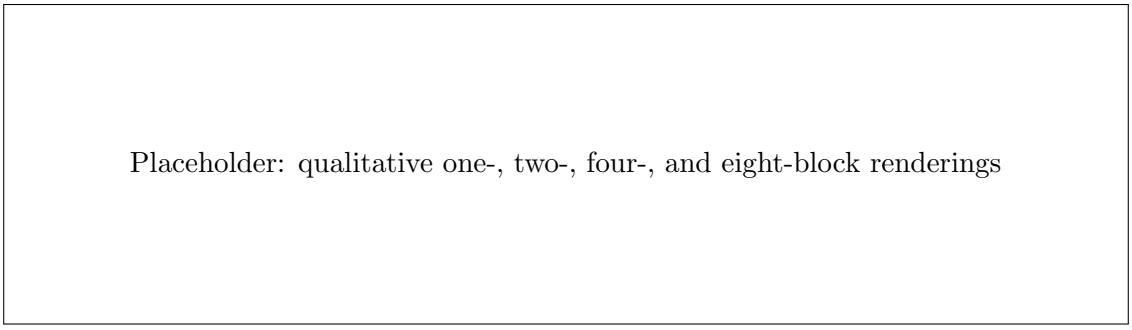

Figure 9: Qualitative rendering examples for different overlap configurations.

6.2 Residual Overlap Penalty

To determine whether overlap introduces a cost beyond payload, the expected render time from the Milestone 2 regression model is compared with the measured time:

$$\Delta T_{\text{overlap}} = T_{\text{measured}} - T_{\text{predicted}}. \tag{31}$$

A positive residual indicates an additional spatial-overlap penalty not explained by payload alone.

Table 8: Measured and predicted render time for overlap configurations.

Case	Predicted Time	Measured Time	Residual	Interpretation
One block	–	–	–	–
Two blocks	–	–	–	–
Four blocks	–	–	–	–
Eight blocks	–	–	–	–

7 Milestone 4: New-Area Loading and Transition Latency

7.1 Objective

This milestone measures the latency introduced when the moving cube enters a region requiring one or more previously inactive blocks.

Two transition conditions are considered:

1. cold transition: the required block is absent from host and GPU caches;
2. warm transition: the block is already cached and only the active set and tile bins must be updated.

7.2 Translation Paths

The evaluated paths include:

1. linear translation through a two-block boundary;
2. diagonal translation through a four-block junction;
3. three-dimensional translation through an eight-block junction;
4. combined translation and orientation changes.

Table 9: Transition paths.

Path	Length	Number of Poses	Transitions	Maximum Active Blocks
Linear boundary	–	–	–	2
Diagonal junction	–	–	–	4
3D junction	–	–	–	8
6DoF navigation	–	–	–	–

7.3 Latency Decomposition

For each transition, we record:

- disk-read time;

- local-to-world transformation time;
- moving-cube Gaussian culling time;
- host merge time;
- host-to-device transfer time;
- tile-preprocessing time;
- scan time;
- radix-sort time;
- tile-range construction time;
- first valid frame latency.

Table 10: New-area transition latency decomposition.

Transition	New MiB	Read	Transform	Cube Cull	H2D	Binning	Sort	First Frame	Total
TODO: Fill	–	–	–	–	–	–	–	–	–

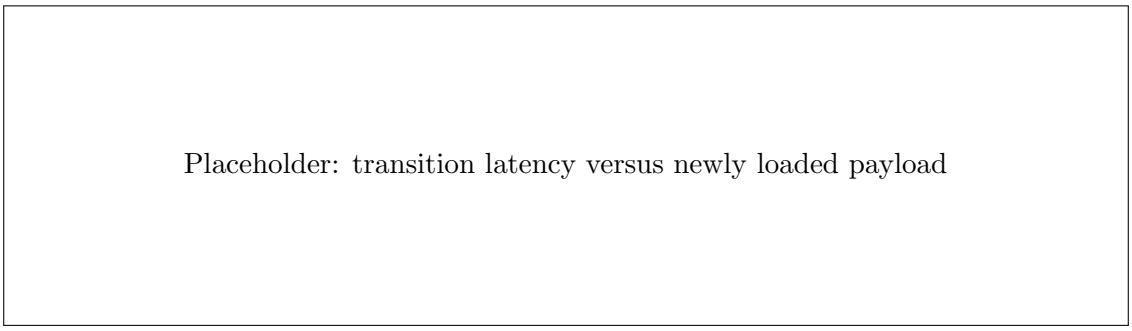

Figure 10: New-area loading latency as a function of newly loaded block payload.

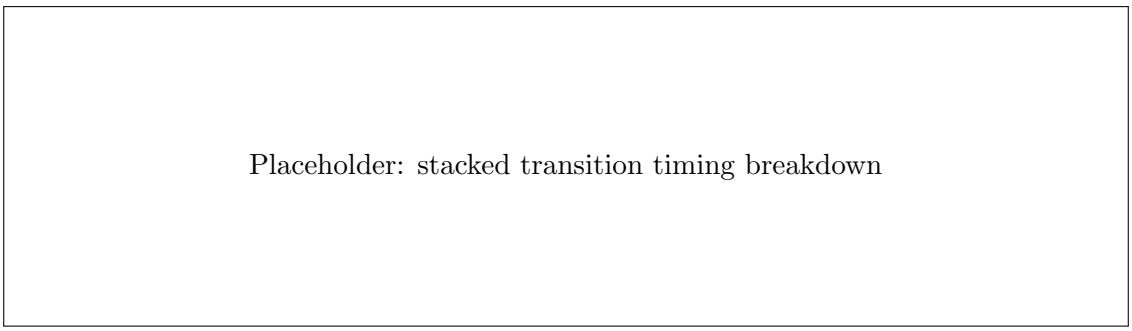

Figure 11: Breakdown of transition latency into loading, transformation, transfer, preprocessing, and rendering components.

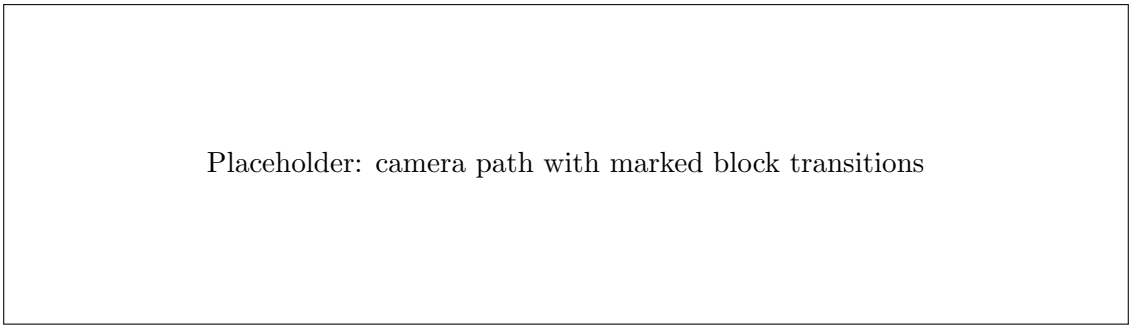

Figure 12: Camera trajectory and locations where the active block set changes.

7.4 Milestone 4 Outcome

This milestone reports:

1. median, p95, and maximum cold-transition latency;
2. median, p95, and maximum warm-transition latency;
3. transition bandwidth;
4. dominant latency component;
5. the number of frames affected by each transition.

TODO: Insert quantitative summary after experiments.

8 Milestone 5: Cross-Hardware Deployment

8.1 Objective

The fifth milestone identifies hardware-normalised metrics that predict whether the renderer sustains at least 30 FPS across different GPU environments.

8.2 Hardware Configuration

For every GPU, record:

- GPU model;
- total VRAM;
- theoretical memory bandwidth;
- compute capability;
- driver version;
- CUDA version;
- power limit and operating mode;
- laptop or desktop deployment.

Table 11: Hardware environments.

GPU	VRAM	Bandwidth	Compute Capability	CUDA	Driver	Power Mode	Platform
TODO: Fill	–	–	–	–	–	–	–

8.3 Reduced Cross-Hardware Matrix

A reduced payload sweep is repeated using

$$\rho \in \{0.10, 0.25, 0.50, 0.75, 1.00\}. \tag{32}$$

The following configurations are evaluated on each GPU:

- one-block interior;
- two-block midpoint;
- four-block junction;
- selected depth-sample counts;
- selected output resolutions.

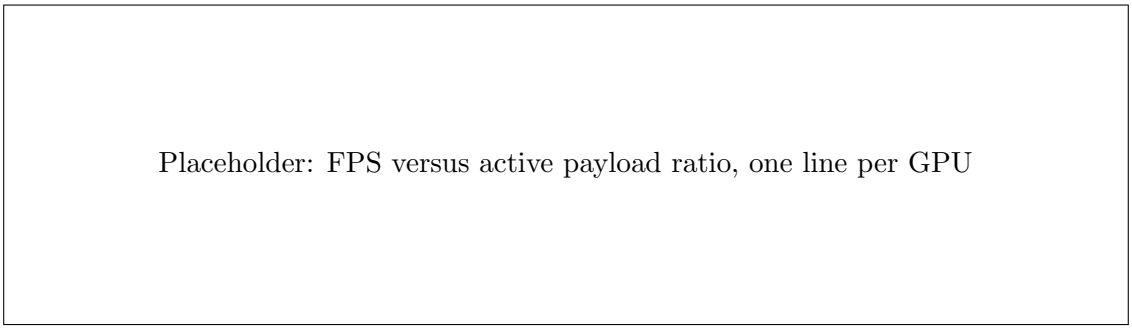

Figure 13: Cross-hardware frame rate as a function of active payload ratio.

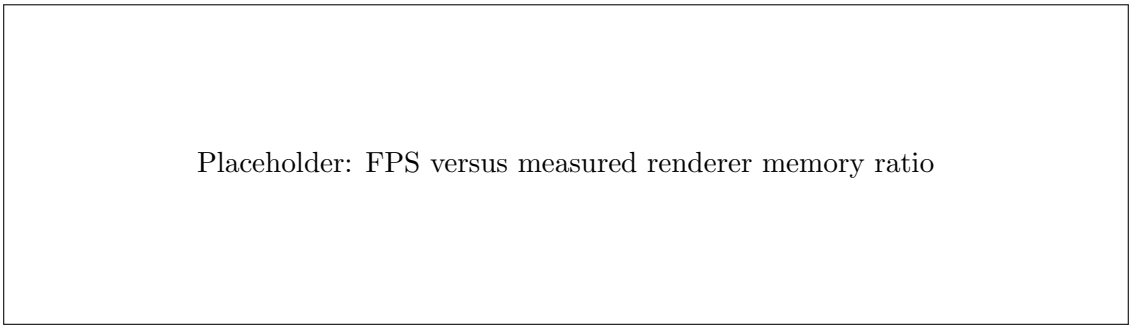

Figure 14: Cross-hardware frame rate as a function of total measured renderer-memory ratio.

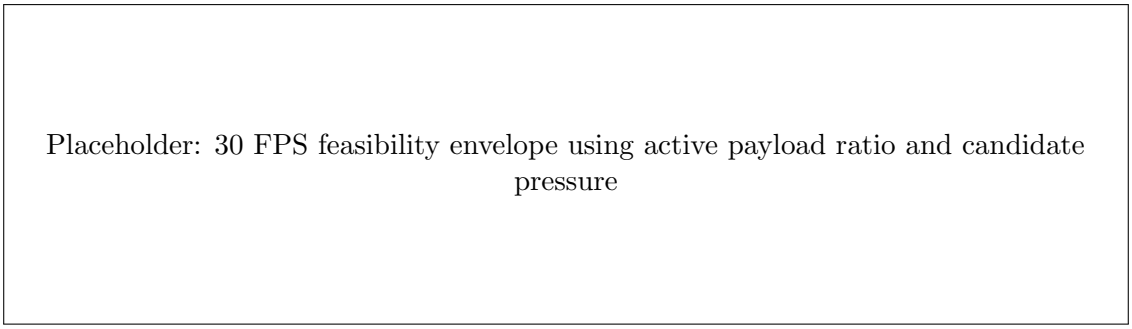

Figure 15: Real-time feasibility envelope. Points are marked as pass or fail according to the 30 FPS criterion.

8.4 Deployment Rule

For each GPU, the selected operating point is the largest configuration satisfying

$$T_{\text{frame},p95} \leq 33.3 \text{ ms}, \tag{33}$$

and

$$R_{\text{renderer}} \leq 0.80. \tag{34}$$

The 20% memory headroom is reserved for transient sort storage, allocator fragmentation, display interoperation, and asynchronous block prefetching.

Table 12: Recommended deployment operating points.

GPU	Resolution	Samples	Cube Edge	Active Blocks	Active Payload	Renderer Ratio	p5 FPS	p95 Transition	Pass
TODO: Fill	–	–	–	–	–	–	–	–	–

9 Milestone 6: Ablation Studies

The following ablations quantify the contribution of individual system components:

1. whole intersecting-block payload versus moving-cube Gaussian culling;
2. frustum culling disabled versus enabled;
3. tile culling disabled versus enabled;
4. Mahalanobis cutoff variation;
5. depth-sample variation;
6. full precision versus half precision;
7. cold loading versus cached loading;
8. synchronous versus asynchronous transfer;
9. no prefetch versus one-block-ahead prefetch.

Table 13: Ablation study results.

Configuration	Active Gaussians	Payload MiB	Pair Count	Setup Time	Render Time	FPS	Memory Ratio	Quality
TODO: Fill	–	–	–	–	–	–	–	–

10 Experimental Protocol

Unless otherwise stated, each configuration uses:

- 20 warm-up frames;
- 300 measured frames;
- five independent timing repetitions;
- three deterministic subset seeds for payload-subsampling experiments.

For each configuration, we report:

- mean;
- median;
- standard deviation;
- minimum and maximum;
- fifth and 95th percentiles;
- 95% confidence intervals where appropriate.

GPU clocks, power mode, display load, and background processes are controlled as far as possible. The first renderer invocation is excluded from steady-state statistics unless cold-start behaviour is explicitly under evaluation.

11 Expected Analysis Structure

The final experimental analysis will be written in the following order:

1. demonstrate numerical correctness;
2. establish the payload-to-FPS relationship;
3. determine whether candidate pressure better explains render time;
4. quantify the additional effect of multi-block overlap;
5. report new-area transition latency;
6. identify hardware-specific real-time operating points;
7. summarise ablation findings.

12 Threats to Validity

Potential threats include:

- independently pretrained blocks may contain inconsistent boundary representations;
- deterministic subsampling changes representation quality as well as payload;
- payload ratio alone does not capture memory bandwidth, cache behaviour, or compute throughput;

- process-based cold-transition timing overestimates integrated persistent-renderer latency;
- Gaussian support-sphere culling is conservative and may retain primitives that never contribute;
- results may depend on volume structure, Gaussian scale distribution, and camera pose.

These threats are addressed using offline merged references, candidate-pressure metrics, repeated trials, multiple camera paths, natural overlap cases, and cross-hardware evaluation.

13 Summary of Milestone Deliverables

Milestone	Main Question	Deliverables
M1	Is the moving-cube multi-block renderer correct?	Single-block equivalence, two-block boundary comparison, timing validation, difference images.
M2	How does FPS change with active Gaussian payload?	Payload sweep, FPS curves, candidate-pressure analysis, regression models, 30 FPS threshold.
M3	How does natural block overlap affect performance?	One-, two-, four-, and eight-block comparisons, residual overlap penalty, qualitative panels.
M4	What is the cost of entering a new area?	Cold/warm transition latency, timing breakdown, bandwidth, camera-path analysis.
M5	How should the renderer be deployed across GPUs?	Cross-hardware curves, feasibility envelope, hardware-specific operating points.
M6	Which components contribute most?	Culling, precision, sampling, caching, transfer, and prefetching ablations.

14 Current Completion Status

Table 15: Current implementation status.

Component	Implemented	Validated	Remaining Work
Moving-cube ray clipping	Yes	Partial	Numerical reference comparison
Multi-block block selection	Yes	Partial	Four- and eight-block cases
Gaussian support culling	Yes	Partial	False-negative validation
Steady-state render timing	Yes	Yes	Repeated-run statistics
Payload and memory ratios	Yes	Partial	Full allocation breakdown
Candidate-pressure recording	Yes	Partial	Regression analysis
Cold transition timing	Partial	No	Direct H2D and bin timing
Persistent GPU cache	No	No	Implementation required
Automatic parameter sweep	No	No	Implementation required
Quality metrics and difference maps	No	No	Implementation required